POO COM RUBY

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

Diferente de outras linguagens onde a Orientação a Objetos é apenas uma opção, no Ruby ela é a essência. Criada para ser "natural, não simples", Ruby foi desenhada para priorizar a produtividade e a “felicidade” do programador.

O que torna o Ruby único?

- POO Pura: Em Ruby, absolutamente tudo é um objeto – desde um número inteiro até uma string ou uma classe.
- Sintaxe Humana: O código é tão limpo e fluido que muitas vezes se lê como uma frase em inglês, reduzindo a carga cognitiva.
- Flexibilidade (Mixins): Através dos Módulos, o Ruby oferece uma forma elegante de compartilhar comportamentos entre classes sem as complicações da herança múltipla.

"Dominar Ruby é entender que a linguagem deve se adaptar ao seu raciocínio, e não o contrário. Vamos transformar lógica em elegância."

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

RELACIONAMENTOS

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

Ou seja:

Composição → dependência total

Agregação → ligação fraca (referência)

CONSTRUTORES E COLECTIONS

1. **Arrays (Coleções Dinâmicas):**
   - O Poder do Array: No Ruby, o Array é a ferramenta principal. Esqueça nomes complexos como "List<T>" ou "Vetores"; o Array já é uma coleção dinâmica por padrão.
   - Tamanho Variável: Você pode adicionar ou remover objetos a qualquer momento sem se preocupar com o tamanho inicial da lista.
   - Polimorfismo Fluido: Graças à tipagem dinâmica, um único Array pode armazenar objetos de diferentes classes filhas de forma totalmente natural, facilitando o processamento de grupos.

INTERFACES

Interfaces em ruby são implementadas via módulos

- Ruby não tem interfaces formais.
- Modules definem contratos de comportamento.
- Classes podem “incluir” módulos com include.

```ruby
module Movimentavel
  def mover
    raise NotImplementedError, "Implementar em classe"
  end
end

class Robo
  include Movimentavel
  def mover; puts "Robo em movimento!"; end
end
```

EXERCÍCIO 1

Cenário: No bar AfterLife, a reputação é tudo. Bebidas comuns têm preço fixo, mas os DrinksIcônicos (homenagens a lendas vivas ou mortas) têm um valor que escala conforme o seu "Nível de Notoriedade" nas ruas.

Tarefa:

1. **Classe Base:** Crie a classe BebidaAfterlife.
   - Defina os atributos **nome** e **custo_base** como protegidos/encapsulados (usando a convenção de underscore).
2. **Encapsulamento**:
   - Para o **nome**, utiliza o “**método**” visto na aula
   - Para o **custo**, implemente **manualmente** um **Getter** (def custo_base) e um Setter (def custo_base).
   - O Setter deve validar se o valor inserido é maior que zero antes de atribuir

EXERCÍCIO 1

- **Herança:** Crie a subclasse DrinkIconico que herda de BebidaAfterlife.
  - Adicione o atributo exclusivo nivel_notoriedade (um número inteiro).
- **Polimorfismo**: Implemente o método preco_total.
  - Na classe base, ele retorna apenas o custo_base.
  - Na classe DrinkIconico, sobrescreva este método para que o cálculo seja:
    - custo_base + (nivel_notoriedade * 8).
- **Representação**: Implemente o método **to_string** em ambas as classes para retornar uma descrição elegante (ex: "Bebida: [Nome] | Valor Final: $[Total]").
- **Na main**: Instancie os **3** objetos de **drinks** com (Nome, Preço Inicial e Notoriedade). em seguida, **calcule** o valor e exiba a descrição final usando uma **lista destes.**

EXERCÍCIO 2

Cenário: Um Maestro coordena uma pequena orquestra. Cada integrante possui uma técnica única, mas todos respondem aos comandos do regente para manter a harmonia. Como podemos observar pela dupla Kosei Arima e Kaori Miyazono.

Tarefa:

- **Abstração**: Crie a classe base **Musico** (nome, instrumento). No método **tocar_partitura**(obra), use raise **NotImplementedError** para forçar a implementação nas classes filhas.
- **Herança**: Crie as classes **Pianista** e **Violinista**. Sobrescreva o método **tocar_partitura**(obra) para imprimir a ação específica de cada instrumento.
  - **Classe Maestro:**
    - **iniciar_concerto**(obra): Recebe o nome da peça e usa .each em uma lista de músicos para chamar o método de tocar a obra de cada um.
    - **ajustar_postura**(estado): Usa .**map** para retornar um novo Array com frases de status da (Ex: "Kaori está em ritmo Allegro!").
- **Main**: Crie sua própria obra. Instancie os músicos, adicione-os ao Maestro e execute tanto o concerto quanto o ajuste de postura.

EXERCÍCIO 3

Cenário: O laboratório de Okabe Rintaro precisa de um sistema simples para registrar seus experimentos. O LabRegistro coordena tudo: ele possui um MicroondasTelefone interno e mantém uma lista de Relatorios de experimentos.

Tarefa:

1. **Composição** (Classe MicroondasTelefone): Crie a classe. Ela deve ter o método ativar que imprime: "Banana Verde detectada! O Microondas-Telefone (nome provisório) está operando!"
2. **Agregação** (Classe Relatorio): Crie a classe **Relatorio** com os atributos **nome_projeto** e **autor**.
3. **Classe LabRegistro:** No **construtor**: Instancie o **MicroondasTelefone** (Composição) e inicie um **array** vazio para guardar os relatórios.
   - Método adicionar_registro: Recebe **nome_projeto** e **autor**. Este método deve:
     i. Chamar o método **ativar** do microondas interno.
     ii. **Criar** um novo objeto **Relatorio** e adicioná-lo ao **array** de registros.
   - Método **listar_experimentos**: varre o **array** de relatórios e imprime o **nome** e o **autor** de cada.
4. **Main**: Instancie o LabRegistro. Chame o método **adicionar_registro** duas ou três vezes com dados diferentes (ex: "KRR", "CERN Hack"). Por fim, chame o método **listar_experimentos** para exibir todos os dados na tela.

EXERCÍCIO 4

Cenário: Sherlock Holmes está investigando um crime em Baker Street e precisa organizar as informações da cena. Cada pessoa presente pode ser uma Testemunha ou um Suspeito, e ele precisa identificar rapidamente quais representam perigo.

Tarefa:

- **Módulos**: Crie a interface, módulo, **Localizável** com o método **local_atual** que imprime a localização da pessoa.
- Crie o módulo **Perigoso** com o método **exibir_ameaca** que imprime uma mensagem de alerta.
- **Classes**: Crie a classe base **Pessoa** com o atributo **nome**. Crie a classe **Testemunha** que herda de **Pessoa** e inclui apenas **Localizavel**. Crie a classe **Suspeito** que herda de **Pessoa** e inclui tanto **Localizavel** quanto **Perigoso**.

EXERCÍCIO 4

- **Agregação**: Crie a classe **CenaDoCrime** que armazena uma **lista** de pessoas. Implemente o método **adicionar_pessoa** para incluir alguém na cena e o método **listar_todos** para exibir todas as pessoas presentes.
- **Filtro com Introspecção**: Na classe **CenaDoCrime**, implemente o método **identificar_perigosos**. Ele deve usar .**select** e **respond_to**?(:exibir_ameaca) para retornar apenas os suspeitos perigosos da lista.
- **Main**: Instancie uma Testemunha e dois Suspeitos com nomes e localizações à sua escolha. Adicione todos à cena, liste todos os presentes e em seguida exiba apenas os identificados como perigosos.

FIM DA AULA!

zSh3rl0cK/**S01-monitoria**

1 Contributor | 0 Issues | 0 Stars | 0 Forks

zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

GitHub